第 3 讲：CSP-S 模考 3

学而思 CSP-S 复赛模考讲评课

倪理涵

学而思素养中心

2026 年 10 月 25 日

题目大意

- 给定操作序列 s，要求按序追加、删除字符，或复制、翻转字符串。
- 求最终字符串的第 k 个字符，或判断无解。



问题分析

- 注意到测试点 $1 \sim 3$：$|s| \leq 10$。
  - $|s|$ 极小，考虑直接模拟执行操作序列。

问题分析

- 注意到测试点 $1 \sim 3$：$|s| \leq 10$。
  - $|s|$ 极小，考虑直接模拟执行操作序列。
- 注意：读到 - 时，字符串非空才操作。

```
1  if (c == '-') {
2    if (!t.empty()) t.pop_back();
3  }
4  else if (c == '*') t += t;
5  else if (c == '!') reverse(t.begin(), t.end());
6  else               t.push_back(c);
```

问题分析

- 注意到测试点 $1 \sim 3$：$|s| \leq 10$。
  - $|s|$ 极小，考虑直接模拟执行操作序列。
- 注意：读到 - 时，字符串非空才操作。

```
1  if (c == '-') {
2    if (!t.empty()) t.pop_back();
3  }
4  else if (c == '*') t += t;
5  else if (c == '!') reverse(t.begin(), t.end());
6  else               t.push_back(c);
```

- 记过程中的字符串最大长度为 L。
- 时间复杂度 $\mathcal{O}(|s|L)$，期望得分 55 分。
  - 多次 * 后，字符串可能极长，无法解决测试点 $4 \sim 8$。

问题分析

- 注意到特殊性质 A：s 中不含 *。
  - 字符串长度始终 $\leq |s|$，仍可考虑模拟执行。

问题分析

- 注意到特殊性质 A：s 中不含 *。
  - 字符串长度始终 $\leq |s|$，仍可考虑模拟执行。
- 注意：读到！ 时，不能再直接 reverse reverse。
  - 翻转操作可能重复出现。

问题分析

- 注意到特殊性质 A：s 中不含 *。
  - 字符串长度始终 $\leq |s|$，仍可考虑模拟执行。
- 注意：读到！ 时，不能再直接 reverse reverse。
  - 翻转操作可能重复出现。
- 可使用 deque 保存字符，并记录操作方向 rev：
  - 对 c：按 rev 决定放到队首还是队尾；
  - 对 -：按 rev 决定从队首还是队尾删除；
  - 对!：只翻转 rev，不移动字符。

问题分析

- 注意到特殊性质 A：s 中不含 *。
  - 字符串长度始终 $\leq |s|$，仍可考虑模拟执行。
- 注意：读到！ 时，不能再直接 reverse reverse。
  - 翻转操作可能重复出现。
- 可使用 deque 保存字符，并记录操作方向 rev：
  - 对 c：按 rev 决定放到队首还是队尾；
  - 对 -：按 rev 决定从队首还是队尾删除；
  - 对!：只翻转 rev，不移动字符。
- 时间复杂度 $O(|s|)$，结合已有部分分，期望得分 30 分。

问题分析

- 注意到特殊性质 B：s 中不含 ！

问题分析

- 注意到特殊性质 B：s 中不含 ！
  - 最终字符串可能很长，应避免保存字符串。

问题分析

- 注意到特殊性质 B：s 中不含 `!`
  - 最终字符串可能很长，应避免保存字符串。
- 逆向思维，从最后一步向前追踪第 k 个字符：
  - 对 c：c 恰为字符串末尾时即为答案，否则化归问题；

问题分析

- 注意到特殊性质 B：s 中不含 !
  - 最终字符串可能很长，应避免保存字符串。
- 逆向思维，从最后一步向前追踪第 k 个字符：
  - 对 c：c 恰为字符串末尾时即为答案，否则化归问题；
  - 对 -：无影响；

问题分析

- 注意到特殊性质 B：s 中不含 !
  - 最终字符串可能很长，应避免保存字符串。
- 逆向思维，从最后一步向前追踪第 k 个字符：
  - 对 c：c 恰为字符串末尾时即为答案，否则化归问题；
  - 对 -：无影响；
  - 对 *：若 k 在后半段，则 k 减去原串长度，化归为前半段的问题。

问题分析

- 注意到特殊性质 B：s 中不含 !
  - 最终字符串可能很长，应避免保存字符串。
- 逆向思维，从最后一步向前追踪第 k 个字符：
  - 对 c：c 恰为字符串末尾时即为答案，否则化归问题；
  - 对 -：无影响；
  - 对 *：若 k 在后半段，则 k 减去原串长度，化归为前半段的问题。
- 为判断无解，以及 c 是否为字符串末尾等，可记录每步操作后的长度 $sz[i]$。

问题分析

- 注意到特殊性质 B：s 中不含 !
  - 最终字符串可能很长，应避免保存字符串。
- 逆向思维，从最后一步向前追踪第 k 个字符：
  - 对 c：c 恰为字符串末尾时即为答案，否则化归问题；
  - 对 -：无影响；
  - 对 *：若 k 在后半段，则 k 减去原串长度，化归为前半段的问题。
- 为判断无解，以及 c 是否为字符串末尾等，可记录每步操作后的长度 $sz[i]$。
- 时间复杂度 $O(|s|)$，结合已有部分分，期望得分 45 分。

问题分析

- 正解只需额外考虑 ！ 的处理。

问题分析

- 正解只需额外考虑 ！ 的处理。
- 求出 $sz[i]$，若最终 $k > sz\left[|s|\right]$，则无解，否则必定有解，从最后一个操作开始逆推。

问题分析

- 正解只需额外考虑 ！ 的处理。
- 求出 $sz[i]$，若最终 $k > sz\left[|s|\right]$，则无解，否则必定有解，从最后一个操作开始逆推。
- 假设操作 i 执行后，字符串长度为 $L = sz[i]$，考虑对其撤销：
  - c：若 $k = L$，则答案为 c，否则继续逆推；
  - $-$：无影响；
  - $*$：若 $k > \frac{L}{2}$，则 $k -= \frac{L}{2}$；

问题分析

- 正解只需额外考虑 ！ 的处理。
- 求出 $sz[i]$，若最终 $k > sz\left[|s|\right]$，则无解，否则必定有解，从最后一个操作开始逆推。
- 假设操作 i 执行后，字符串长度为 $L = sz[i]$，考虑对其撤销：
  - c：若 $k = L$，则答案为 c，否则继续逆推；
  - $-$：无影响；
  - $*$：若 $k > \frac{L}{2}$，则 $k \mathrel{-}= \frac{L}{2}$；
  - $!$：位置关于中心镜像，令 $k \leftarrow L - k + 1$。

问题分析

- 正解只需额外考虑 ！ 的处理。
- 求出 $sz[i]$，若最终 $k > sz\left[|s|\right]$，则无解，否则必定有解，从最后一个操作开始逆推。
- 假设操作 i 执行后，字符串长度为 $L = sz[i]$，考虑对其撤销：
  - c：若 $k = L$，则答案为 c，否则继续逆推；
  - $-$：无影响；
  - $*$：若 $k > \frac{L}{2}$，则 $k \mathrel{-}= \frac{L}{2}$；
  - $!$：位置关于中心镜像，令 $k \leftarrow L - k + 1$。
- 时间复杂度 $O(|s|)$，期望得分 100 分。

题目大意

- 有 n 张牌，初始时王牌位置为 p，m 个位置锁定。
- 每次操作可翻转一段长为 k 的区间，但不能将王牌移入锁定位置。
- 求王牌移到位置 $1,\ldots,n$ 的最少操作次数，或判断无解。



问题分析

- 考虑翻转 $[l, l+k-1]$ 的影响。

问题分析

- 考虑翻转 $[l, l+k-1]$ 的影响。
  - 对区间中的位置 i，其纸牌翻转后的位置应为：
    $$l + (l+k-1) - i = 2l + k - 1 - i$$
  - 为使 i 落在区间中，l 可取 $i-k+1, \ldots, i$；
  - 依次计算，可得 i 按不同方式翻转后的落点。

问题分析

- 考虑翻转 $[l, l+k-1]$ 的影响。
  - 对区间中的位置 i，其纸牌翻转后的位置应为：
    $$l + (l+k-1) - i = 2l + k - 1 - i$$
    - 为使 i 落在区间中，l 可取 $i-k+1, \ldots, i$；
    - 依次计算，可得 i 按不同方式翻转后的落点。
- 注意：翻转时，还应确保翻转区间存在。

问题分析

- 考虑翻转 $[l, l+k-1]$ 的影响。
  - 对区间中的位置 i，其纸牌翻转后的位置应为：
    $$l + (l+k-1) - i = 2l + k - 1 - i$$
  - 为使 i 落在区间中，l 可取 $i-k+1, \ldots, i$；
  - 依次计算，可得 i 按不同方式翻转后的落点。
- 注意：翻转时，还应确保翻转区间存在。
  - 记 l 的取值范围为 $[L, R]$，则：

    $$L = \max(i-k+1, 1), \quad R = \min(i, n-k+1)$$

问题分析

- 考虑翻转 $[l, l+k-1]$ 的影响。
  - 对区间中的位置 i，其纸牌翻转后的位置应为：
    $$l + (l+k-1) - i = 2l + k - 1 - i$$
  - 为使 i 落在区间中，l 可取 $i-k+1, \ldots, i$；
  - 依次计算，可得 i 按不同方式翻转后的落点。
- 注意：翻转时，还应确保翻转区间存在。
  - 记 l 的取值范围为 $[L, R]$，则：
    $$L = \max(i-k+1, 1), \quad R = \min(i, n-k+1)$$
- 理解：若一次翻转能从 i 移到 j，则连边 $i \to j$。
- 问题即求：从 p 出发，到各点的最短路。
  - 边权为 1，可使用 BFS 求解。

问题分析

- 考虑翻转 $[l, l+k-1]$ 的影响。
  - 对区间中的位置 i，其纸牌翻转后的位置应为：
    $$l + (l+k-1) - i = 2l + k - 1 - i$$
    - 为使 i 落在区间中，l 可取 $i-k+1, \ldots, i$；
    - 依次计算，可得 i 按不同方式翻转后的落点。
- 注意：翻转时，还应确保翻转区间存在。
  - 记 l 的取值范围为 $[L, R]$，则：
    $$L = \max(i-k+1, 1), \quad R = \min(i, n-k+1)$$
- 理解：若一次翻转能从 i 移到 j，则连边 $i \to j$。
- 问题即求：从 p 出发，到各点的最短路。
  - 边权为 1，可使用 BFS 求解。
- 时间复杂度 $\mathcal{O}(nk)$，期望得分 64 分。

问题分析

- 即使没有想到 BFS，也可以手做一些特殊数据。

问题分析

- 即使没有想到 BFS，也可以手做一些特殊数据。
- 注意到测试点 11：$k = 1$。

问题分析

- 即使没有想到 BFS，也可以手做一些特殊数据。
- 注意到测试点 11：$k = 1$。
  - 翻转区间不改变位置，答案为 0 或 -1。

问题分析

- 即使没有想到 BFS，也可以手做一些特殊数据。
- 注意到测试点 11：$k = 1$。
  - 翻转区间不改变位置，答案为 0 或 -1。
- 注意到测试点 $12 \sim 14$：$k = 2, 3$。

问题分析

- 即使没有想到 BFS，也可以手做一些特殊数据。
- 注意到测试点 11：$k = 1$。
  - 翻转区间不改变位置，答案为 0 或 -1。
- 注意到测试点 $12 \sim 14$：$k = 2, 3$。
  - $k = 2$ 时，翻转等效于左、右移；
  - $k = 3$ 时，翻转等效于左、右跳 2 格；
  - 只需线性扫描即可解决。

问题分析

- 即使没有想到 BFS，也可以手做一些特殊数据。
- 注意到测试点 11：$k = 1$。
  - 翻转区间不改变位置，答案为 0 或 -1。
- 注意到测试点 $12 \sim 14$：$k = 2, 3$。
  - $k = 2$ 时，翻转等效于左、右移；
  - $k = 3$ 时，翻转等效于左、右跳 2 格；
  - 只需线性扫描即可解决。
- 注意到测试点 $15 \sim 16$：$k = n, n - 1$。
  - $k = n$ 时，只有一个可翻转区间；
  - $k = n - 1$ 时，只有两个区间，有限步枚举即可。

问题分析

CSP-S 模考 3

倪理涵

T1, T2
图灵机
翻转纸牌

T3, T4
未来程序
计算

- l 每增加 1，落点 $j = 2l + k - 1 - i$ 增加 2。

问题分析

- l 每增加 1，落点 $j = 2l + k - 1 - i$ 增加 2。
  - 理解：后继位置的奇偶性固定。

问题分析

- l 每增加 1，落点 $j = 2l + k - 1 - i$ 增加 2。
  - 理解：后继位置的奇偶性固定。
- 枚举时，状态 i 的全部后继 j，恰为该区间内符合某一奇偶性的未锁定位置。
  - 每个位置应只在 BFS 中被枚举到一次，才能保证复杂度。

问题分析

- l 每增加 1，落点 $j = 2l + k - 1 - i$ 增加 2。
  - 理解：后继位置的奇偶性固定。
- 枚举时，状态 i 的全部后继 j，恰为该区间内符合某一奇偶性的未锁定位置。
  - 每个位置应只在 BFS 中被枚举到一次，才能保证复杂度。
- 可使用 set，分别维护奇数、偶数位置。

问题分析

- l 每增加 1，落点 $j = 2l + k - 1 - i$ 增加 2。
  - 理解：后继位置的奇偶性固定。
- 枚举时，状态 i 的全部后继 j，恰为该区间内符合某一奇偶性的未锁定位置。
  - 每个位置应只在 BFS 中被枚举到一次，才能保证复杂度。
- 可使用 set，分别维护奇数、偶数位置。
  - 初始时，放入所有未锁定的位置；
  - 出发点 p 无需进入集合。

问题分析

- BFS 时，仍求出 i 的对应后继区间，及目标奇偶性。
- 在对应集合中，反复查找不小于 L 的最小位置 pos：
  - 若不超过 R，则更新最短路、入队；
  - 最后，还应从 set 中删除 pos；

问题分析

- BFS 时，仍求出 i 的对应后继区间，及目标奇偶性。
- 在对应集合中，反复查找不小于 L 的最小位置 pos：
  - 若不超过 R，则更新最短路、入队；
  - 最后，还应从 set 中删除 pos；
- 每个位置最多入队、删除一次，每次集合操作为 $O(\log n)$。

问题分析

- BFS 时，仍求出 i 的对应后继区间，及目标奇偶性。
- 在对应集合中，反复查找不小于 L 的最小位置 pos：
  - 若不超过 R，则更新最短路、入队；
  - 最后，还应从 set 中删除 pos；
- 每个位置最多入队、删除一次，每次集合操作为 $O(\log n)$。
- 时间复杂度 $O(n \log n)$，期望得分 100 分。

题目大意

- 初始数组为 $a_1, \ldots, a_n$；每次操作 (l, r, k, v)，将 $l, l+k, l+2k, \ldots \leq r$ 位置乘 v 并模 10^9+7。
- 所有操作结束后，输出数组元素的按位异或和。
- 直接执行每次操作可能重复修改大量位置，需要利用步长 k 的结构。

问题分析

- 最直接的方法是按操作遍历 $i = l, l+k, l+2k, \ldots \le r$，并执行 $a_i \leftarrow a_i v \bmod MOD$。
- 最后扫描数组求异或，暴力复杂度为 $O(n + \sum n/k)$。

问题分析

- 最直接的方法是按操作遍历 $i = l, l+k, l+2k, \ldots \le r$，并执行 $a_i \leftarrow a_i v \bmod MOD$。
- 最后扫描数组求异或，暴力复杂度为 $O(n + \sum n/k)$。
- 测试点 $1 \sim 2$：$n, q \le 1000$ 且 $v \le 5$，直接模拟可得 10 分。
- 测试点 3：$n, q \le 1000$，无性质，直接模拟可得 5 分。
- 但完整数据有 $n, q \le 2 \times 10^5$，反复修改会超时。

问题分析

- 测试点 $4 \sim 7$：每次操作均满足 $k > \lfloor \sqrt{n} \rfloor$。
- 一次操作最多修改 $\lceil n/k \rceil = O(\sqrt{n})$ 个位置，因此仍可逐个更新。

问题分析

- 测试点 $4 \sim 7$：每次操作均满足 $k > \lfloor \sqrt{n} \rfloor$。
- 一次操作最多修改 $\lceil n/k \rceil = O(\sqrt{n})$ 个位置，因此仍可逐个更新。
- 所有操作总计 $O(q\sqrt{n})$ 次更新，最后 $O(n)$ 求异或。
- 这一档得 20 分。注意优化依据是步长大，而不是乘数 v 的大小。

问题分析

- 测试点 $8 \sim 11$：每次操作 $k = 1$，修改范围就是普通区间 $[l, r]$。
- 所有操作都是乘法，可先汇总每个位置受到的累计乘数，再乘回初始 a_i。

问题分析

- 测试点 $8 \sim 11$：每次操作 $k = 1$，修改范围就是普通区间 $[l, r]$。
- 所有操作都是乘法，可先汇总每个位置受到的累计乘数，再乘回初始 a_i。
- 维护乘法差分：区间左端乘 v，右端后一位乘 v^{-1}；做前缀乘积即可得到每个位置的累计乘数。
- 因 $MOD = 10^9 + 7$ 为质数且 $1 \le v < MOD$，有 $v^{-1} \equiv v^{MOD-2} \pmod{MOD}$。
- 时间 $O(n + q)$ 次差分与扫描，快速幂求逆元另需 $O(q \log MOD)$；本档得 20 分。

问题分析

- 以普通位置序号表示一条等差数列。若只希望区间 $[L, R]$ 乘 v，就在 L 乘 v，在 $R+1$ 乘 v^{-1}。
- 前缀乘积在 $L \sim R$ 期间多出一个 v；经过 $R+1$ 后，v 与 v^{-1} 抵消。

问题分析

- 以普通位置序号表示一条等差数列。若只希望区间 $[L,R]$ 乘 v，就在 L 乘 v，在 $R+1$ 乘 v^{-1}。
- 前缀乘积在 $L\sim R$ 期间多出一个 v；经过 $R+1$ 后，v 与 v^{-1} 抵消。
- 因此差分项不是加法，而是模意义下的乘法；还原方式是做前缀乘积。
- 当 $k=1$ 时，这就是普通区间乘法差分；一般 k 时，则要先按步长和余数拆成等差数列。

问题分析

- 测试点 $12 \sim 16$：每次操作满足 $v \leq 5$，共 25 分。
- 可预先计算 $1, 2, 3, 4, 5$ 的逆元，避免为这些乘数重复快速幂。
- 但 v 小并不意味着操作影响位置少：当 $k = 1$ 时一次操作仍可能修改 $\Theta(n)$ 个元素。

问题分析

- 测试点 $12 \sim 16$：每次操作满足 $v \leq 5$，共 25 分。
- 可预先计算 $1, 2, 3, 4, 5$ 的逆元，避免为这些乘数重复快速幂。
- 但 v 小并不意味着操作影响位置少：当 $k = 1$ 时一次操作仍可能修改 $\Theta(n)$ 个元素。
- 因此不能仅凭性质 C 就把所有操作逐项展开；需要进一步按 k 处理，完整解法对任意 v 都适用。

问题分析

- 取阈值 $B = \lfloor \sqrt{n} \rfloor + 1$，把操作分为 $k > B$ 与 $k \leq B$。
- 若 $k > B$，本次至多影响 $\lceil n/k \rceil = O(n/B) = O(\sqrt{n})$ 个元素。
- 直接遍历这些下标并更新数组，每个大步长操作耗时 $O(\sqrt{n})$，总计 $O(q\sqrt{n})$。

问题分析

- 取阈值 $B = \lfloor \sqrt{n} \rfloor + 1$，把操作分为 $k > B$ 与 $k \leq B$。
- 若 $k > B$，本次至多影响 $\lceil n/k \rceil = O(n/B) = O(\sqrt{n})$ 个元素。
- 直接遍历这些下标并更新数组，每个大步长操作耗时 $O(\sqrt{n})$，总计 $O(q\sqrt{n})$。
- 大步长的关键：单次操作覆盖的点很少；小步长则反过来批量处理。

问题分析

- 固定小步长 k 后，按 $r_0 = l \bmod k$ 分组；组内可能被修改的下标为

$$r_0,\ r_0 + k,\ r_0 + 2k, \ldots.$$

- 把这些位置换成序号 $0, 1, 2, \ldots$ 后，原来的等差数列区间操作变成序列上的连续区间乘法。
- 对区间端点做乘法差分，最后在该余数序列做前缀乘积，批量更新原数组。

问题分析

- 固定小步长 k 后，按 $r_0 = l \bmod k$ 分组；组内可能被修改的下标为

$$r_0,\ r_0 + k,\ r_0 + 2k, \ldots.$$

- 把这些位置换成序号 $0, 1, 2, \ldots$ 后，原来的等差数列区间操作变成序列上的连续区间乘法。
- 对区间端点做乘法差分，最后在该余数序列做前缀乘积，批量更新原数组。
- 固定一个 k，所有余数序列长度之和为 $O(n)$；小步长种类 $O(\sqrt{n})$，处理扫描总计 $O(n\sqrt{n})$。

问题分析

- 测试点 $17 \sim 20$ 无特殊性质：将大步长操作逐项更新，小步长操作按 $(k, l \bmod k)$ 分组做乘法差分。
- 大步长耗时 $O(q\sqrt{n})$；小步长扫描耗时 $O(n\sqrt{n})$，每个操作的分组/端点更新为 $O(q)$。

问题分析

- 测试点 $17 \sim 20$ 无特殊性质：将大步长操作逐项更新，小步长操作按 $(k, l \bmod k)$ 分组做乘法差分。
- 大步长耗时 $O(q\sqrt{n})$；小步长扫描耗时 $O(n\sqrt{n})$，每个操作的分组/端点更新为 $O(q)$。
- 总时间复杂度为 $O((n+q)\sqrt{n})$；若每次端点都用快速幂求逆元，再额外计入 $O(q \log MOD)$。
- 处理完所有操作后，对每个 a_i 乘累计乘数，再扫描求异或；空间复杂度 $O(n+q)$。

题目大意

- 给定 n 个点的树，根结点为 1。
- 叶结点为输入结点，其他点为计算结点。
- 所有叶子先同时占用 w_i 大小的空间；计算节点必须等儿子都完成后才能计算。
- 计算节点先申请 w_x 的新空间，再释放所有儿子的空间。
- 可调整合法的计算顺序，求最小可能的峰值内存。

问题分析

- 所有叶子输入在计算开始前同时存在，构成固定的初始内存基线：

$$W_{leaf} = \sum_{i \text{ 是叶子}} w_i.$$

- 之后每完成一个计算节点，儿子结果空间释放，节点自身结果保留，等待父节点使用。

问题分析

- 所有叶子输入在计算开始前同时存在，构成固定的初始内存基线：

$$W_{leaf} = \sum_{i \text{ 是叶子}} w_i.$$

- 之后每完成一个计算节点，儿子结果空间释放，节点自身结果保留，等待父节点使用。
- 因此，计算顺序会改变峰值：
  - 某个子树过早完成，它的结果可能在内存中等待较久；
  - 而计算父节点时，新旧结果还会短暂共存。

问题分析

- 所有叶子输入在计算开始前同时存在，构成固定的初始内存基线：

$$W_{leaf} = \sum_{i \text{ 是叶子}} w_i.$$

- 之后每完成一个计算节点，儿子结果空间释放，节点自身结果保留，等待父节点使用。
- 因此，计算顺序会改变峰值：
  - 某个子树过早完成，它的结果可能在内存中等待较久；
  - 而计算父节点时，新旧结果还会短暂共存。
- 考虑比较计算过程的“峰值需求”和“结束后留下的内存”。

问题分析

- 注意到测试点 $1 \sim 6$：$n \leq 10$。
  - 可使用状态压缩，枚举已完成计算的节点集合。

问题分析

- 注意到测试点 $1 \sim 6$：$n \leq 10$。
  - 可使用状态压缩，枚举已完成计算的节点集合。
- $dp[S]$：完成 S 中计算节点时的最小峰值。

问题分析

- 注意到测试点 $1 \sim 6$：$n \leq 10$。
  - 可使用状态压缩，枚举已完成计算的节点集合。
- $dp[S]$：完成 S 中计算节点时的最小峰值。
- 若节点 $x \notin S$ 的所有儿子均为输入叶子或已在 S 中，则 x 当前可计算。

问题分析

- 注意到测试点 $1 \sim 6$：$n \leq 10$。
  - 可使用状态压缩，枚举已完成计算的节点集合。
- $dp[S]$：完成 S 中计算节点时的最小峰值。
- 若节点 $x \notin S$ 的所有儿子均为输入叶子或已在 S 中，则 x 当前可计算。
- 当前常驻内存由 S 唯一确定：从 W_{leaf} 加上已完成节点权值，再减去已释放的儿子权值，得到 $M(S)$。

问题分析

- 注意到测试点 $1 \sim 6$：$n \leq 10$。
  - 可使用状态压缩，枚举已完成计算的节点集合。
- $dp[S]$：完成 S 中计算节点时的最小峰值。
- 若节点 $x \notin S$ 的所有儿子均为输入叶子或已在 S 中，则 x 当前可计算。
- 当前常驻内存由 S 唯一确定：从 W_{leaf} 加上已完成节点权值，再减去已释放的儿子权值，得到 $M(S)$。
- 状态转移方程：若 x 的儿子均已完成或为叶子，则：

$$dp[S \cup \{x\}] = \min\left(dp[S \cup \{x\}], \max(dp[S], M(S) + w_x)\right).$$

问题分析

- 注意到测试点 $1 \sim 6$：$n \leq 10$。
  - 可使用状态压缩，枚举已完成计算的节点集合。
- $dp[S]$：完成 S 中计算节点时的最小峰值。
- 若节点 $x \notin S$ 的所有儿子均为输入叶子或已在 S 中，则 x 当前可计算。
- 当前常驻内存由 S 唯一确定：从 W_{leaf} 加上已完成节点权值，再减去已释放的儿子权值，得到 $M(S)$。
- 状态转移方程：若 x 的儿子均已完成或为叶子，则：

$$dp[S \cup \{x\}] = \min\left(dp[S \cup \{x\}], \max(dp[S], M(S) + w_x)\right).$$

- 时间复杂度 $\mathcal{O}(n2^n)$，期望得分 30 分。

问题分析

- 一段连续计算过程可表示为 (al, fe)：
  - al 是相对开始内存的最大额外占用；
  - $al - fe$ 是过程结束时的内存变化量。

问题分析

- 一段连续计算过程可表示为 (al, fe)：
  - al 是相对开始内存的最大额外占用；
  - $al - fe$ 是过程结束时的内存变化量。
- 单个计算节点 i：计算时申请 w_i，完成后释放所有儿子结果，故可记为：

$$(al_i, fe_i) = \left(w_i, \sum_{c \text{ 是} i \text{ 的儿子}} w_c \right).$$

问题分析

- 一段连续计算过程可表示为 (al, fe)：
  - al 是相对开始内存的最大额外占用；
  - $al - fe$ 是过程结束时的内存变化量。
- 单个计算节点 i：计算时申请 w_i，完成后释放所有儿子结果，故可记为：

$$(al_i, fe_i) = \left(w_i, \sum_{c \text{ 是} i \text{ 的儿子}} w_c \right).$$

- 若先执行过程 x 再执行过程 y，第二段开始时已有净变化 $al_x - fe_x$。

问题分析

- 若先执行过程 x 再执行过程 y，第二段开始时已有净变化 $al_x - fe_x$。

问题分析

- 若先执行过程 x 再执行过程 y，第二段开始时已有净变化 $al_x - fe_x$。
- 合并后的峰值额外占用为

$$al' = \max(al_x,\ al_x - fe_x + al_y).$$

- 合并结束的净变化为 $(al_x - fe_x) + (al_y - fe_y)$，因此

$$fe' = al' - (al_x - fe_x + al_y - fe_y).$$

问题分析

- 若先执行过程 x 再执行过程 y，第二段开始时已有净变化 $al_x - fe_x$。
- 合并后的峰值额外占用为

$$al' = \max(al_x,\ al_x - fe_x + al_y).$$

- 合并结束的净变化为 $(al_x - fe_x) + (al_y - fe_y)$，因此

$$fe' = al' - (al_x - fe_x + al_y - fe_y).$$

- 同一个父节点的儿子子树可按不同顺序完成。
- 先后顺序影响尚未释放的结果空间，也影响峰值。

问题分析

- 贪心：使用 (al, fe) 比较优先级。
  - 每次选出优先队列中当前最优先的可合并组，和它的父节点组连接。

问题分析

- 贪心：使用 (al, fe) 比较优先级。
  - 每次选出优先队列中当前最优先的可合并组，和它的父节点组连接。
- 合并时，把子组过程接到父组过程之前，并用合并公式更新父组的 (al, fe)：

```
1 val[y] = merge(val[x],val[y]);
```

问题分析

- 贪心：使用 (al, fe) 比较优先级。
  - 每次选出优先队列中当前最优先的可合并组，和它的父节点组连接。
- 合并时，把子组过程接到父组过程之前，并用合并公式更新父组的 (al, fe)：

```
1 val[y] = merge(val[x],val[y]);
```

- 注意：
  - 合并会改变父组优先级，更新后应重新入堆；

问题分析

- 贪心：使用 (al, fe) 比较优先级。
  - 每次选出优先队列中当前最优先的可合并组，和它的父节点组连接。
- 合并时，把子组过程接到父组过程之前，并用合并公式更新父组的 (al, fe)：

```
1 val[y] = merge(val[x],val[y]);
```

- 注意：
  - 合并会改变父组优先级，更新后应重新入堆；
  - 堆中旧条目可能已过期，弹出时检查代表元及 (al, fe) 是否仍与当前状态一致。

问题分析

- 贪心：使用 (al, fe) 比较优先级。
  - 每次选出优先队列中当前最优先的可合并组，和它的父节点组连接。
- 合并时，把子组过程接到父组过程之前，并用合并公式更新父组的 (al, fe)：

```
1 val[y] = merge(val[x],val[y]);
```

- 注意：
  - 合并会改变父组优先级，更新后应重新入堆；
  - 堆中旧条目可能已过期，弹出时检查代表元及 (al, fe) 是否仍与当前状态一致。
- 时间复杂度 $O(n \log n)$，期望得分 100 分。

Thanks for your attention!